

Arbori de selectie (Heap)

Memoreaza elementele conform unei **relatii de ordine** definite pe multimea cheilor si asigura **extragerea rapida** a elementelor in ordinea dictata de aceasta relatie.

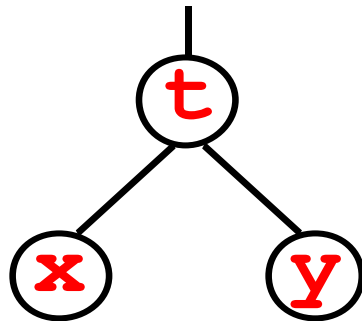
➤ tratarea cererilor de acordare a unei burse in **ordinea descrescatoare a mediei** generale a solicitantilor

In literatura se folosesc si termenii de **Heap** (nu are legatura cu zona de memorie destinata alocarii dinamice), **arbore partial ordonat**, **coada de prioritati**.

Arbori de selectie (Heap)

Caracteristici:

- este arbore binar de **inaltime minima** ($\log n$)
- **inserarea** se realizeaza pe principiul parcurgerii in latime
→ **ultimul nivel este completat de la stanga spre dreapta**
- pentru **orice pereche de noduri tata-fiu** cheile **respecta relatia de ordine prestabilita** (cheia nodului tata este predecesoare sau echivalenta cu cele ale nodurilor fiu)



$$\text{Rel}(t, x) \leq 0$$

$$\text{Rel}(t, y) \leq 0$$

■ qsort

- $v, nv, \dim \text{ elem}, \text{fcmp}$

- $\text{fcmp}(a,b)$:

 - 0 : a echiv b

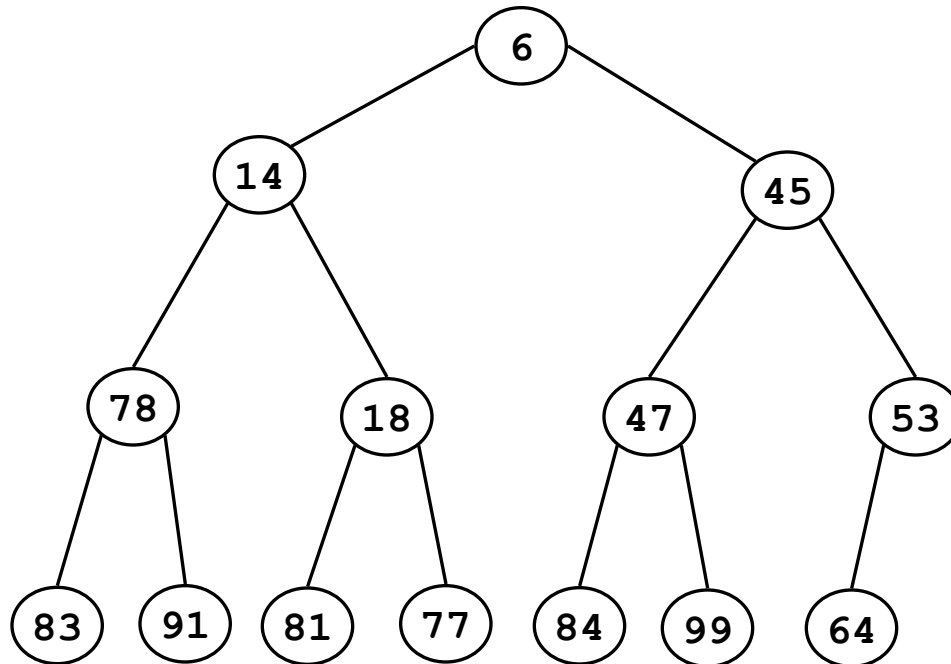
 - < 0 : a, b

 - > 0 : b, a

- Min-heap: $\text{info}(\text{parinte}) < \text{info}(\text{st}), \text{info}(\text{dr})$

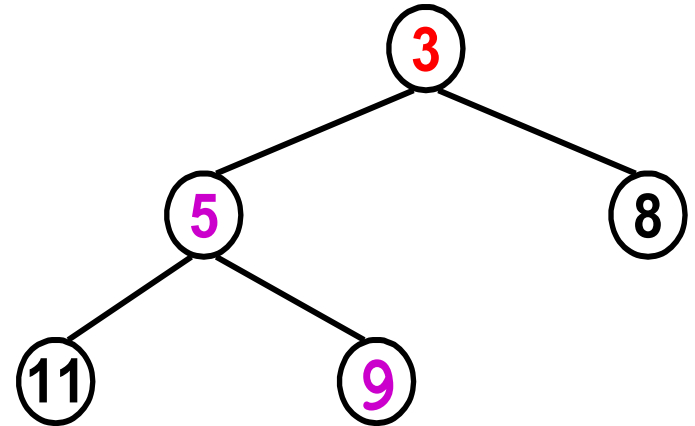
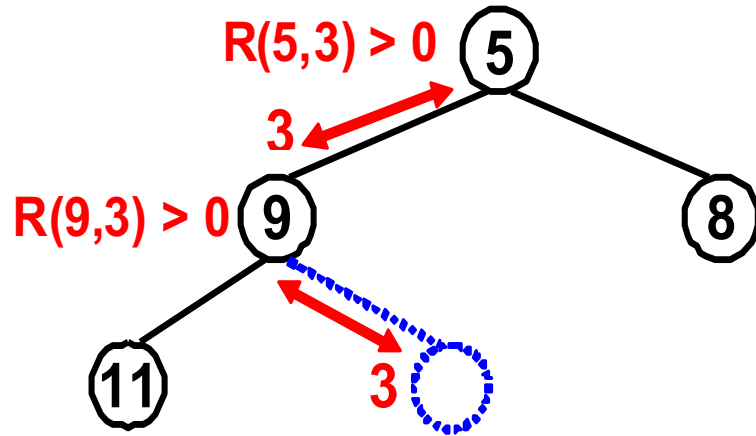
- Max-heap: $\text{info}(\text{parinte}) > \text{info}(\text{st}), \text{info}(\text{dr})$

Exemplu: heap (min-heap)

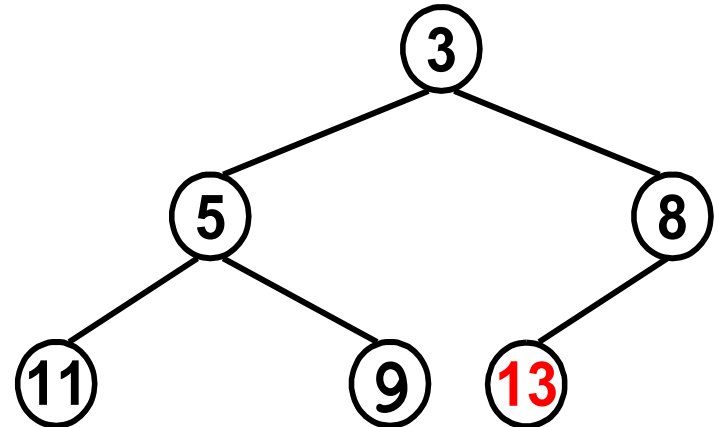
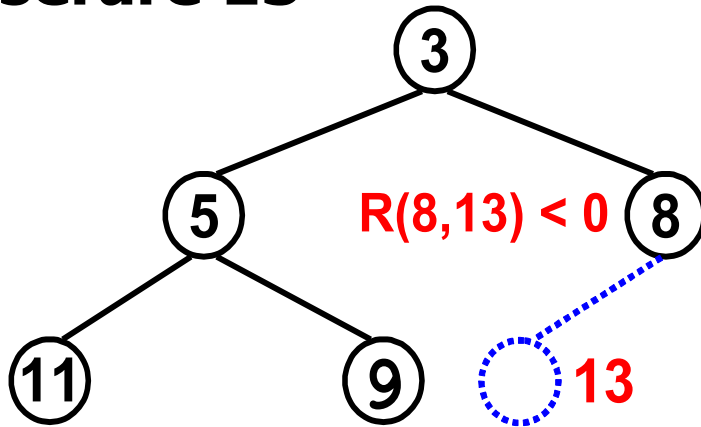


Inserare in heap (min-heap)

Inserare 3

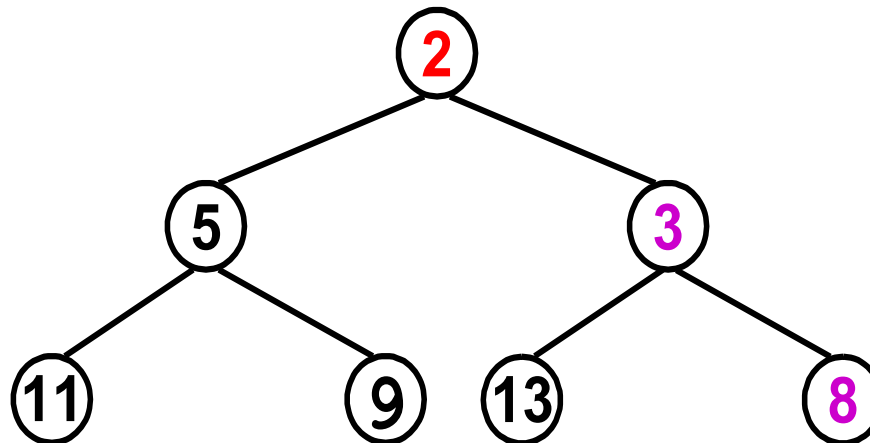
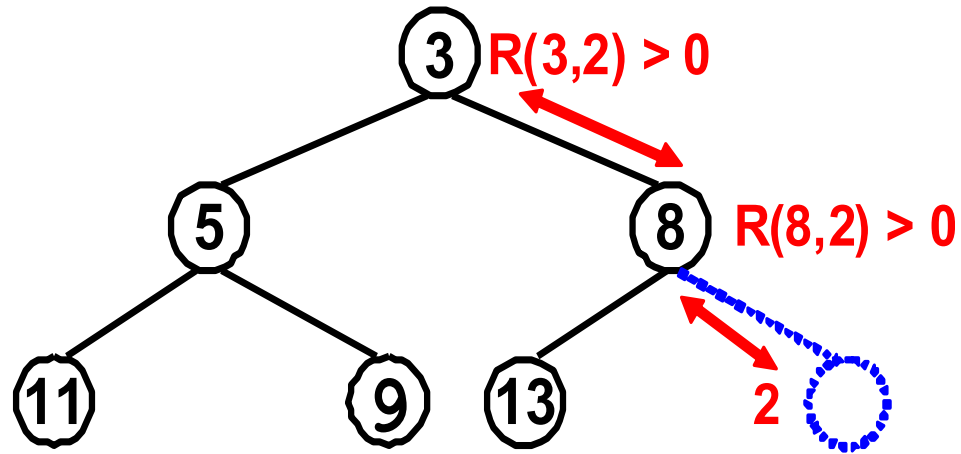


Inserare 13

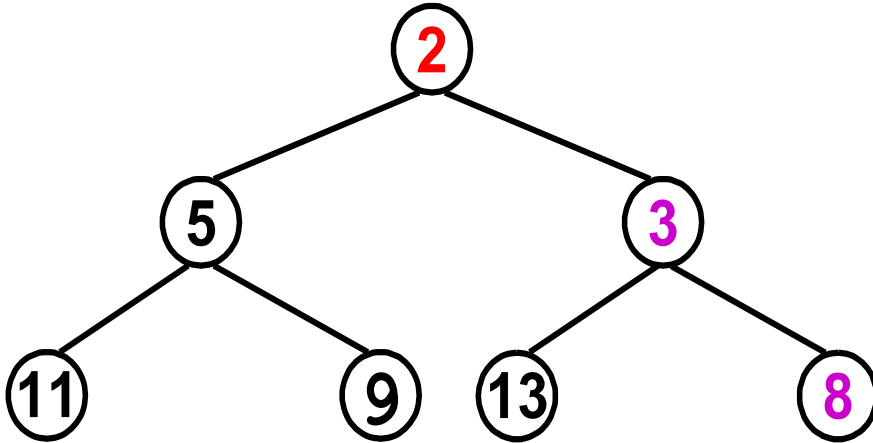


Inserare in heap (min-heap)

Inserare 2



Insertare in heap (min-heap)



Algoritm inserare in heap

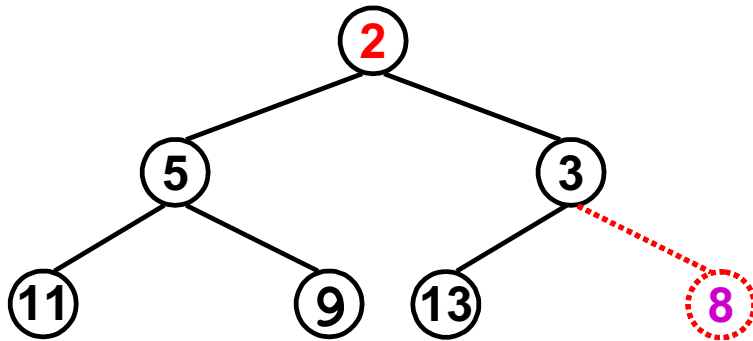
Fie **x** informatia ce trebuie inserata in arborele A

```
{ adauga in A nod Nou, fara informatie;  
  D = Nou;  
  cat timp (D != Radacina(A) si  
             Rel(cheie(Tata(D)), cheie(x)) > 0)  
  { transfer info(Tata(D)) in D;  
    D = Tata(D);  
  }  
  transfera x in nodul D;  
}
```


Extragere din heap

- Se extrage informatia din radacina
- Se elimina nodul din extrema dreapta a ultimului nivel.
- Informatia din nodul eliminat ramane in radacina sau, daca este cazul, se propaga spre nivelurile inferioare, pentru a pastra organizarea de arbore de selectie.

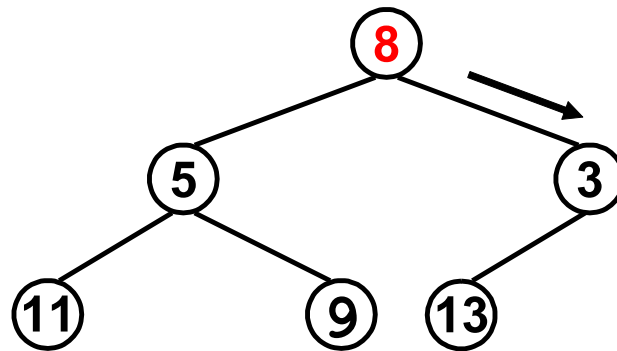
Eliminare din heap (min-heap)



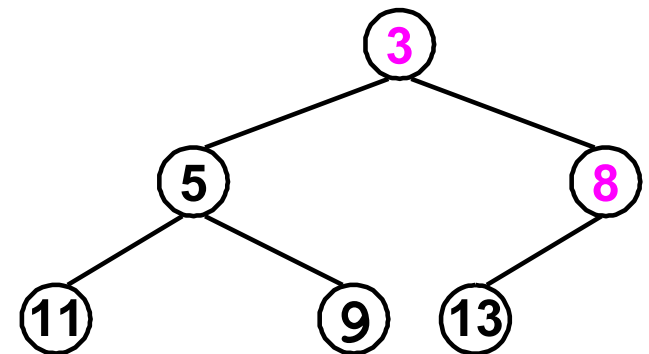
Extras = 2

Temp = 8

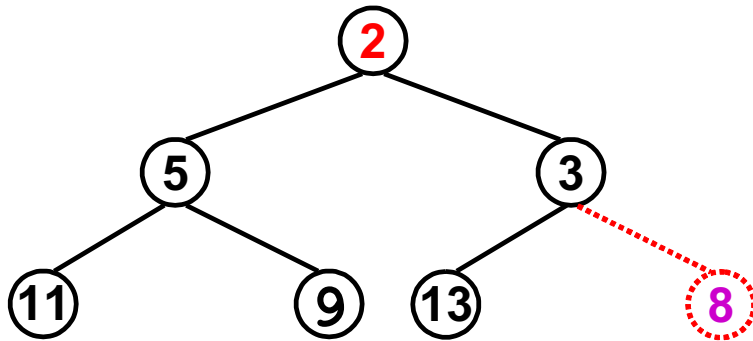
Elim val din rad



$R(3, 8) < 0$



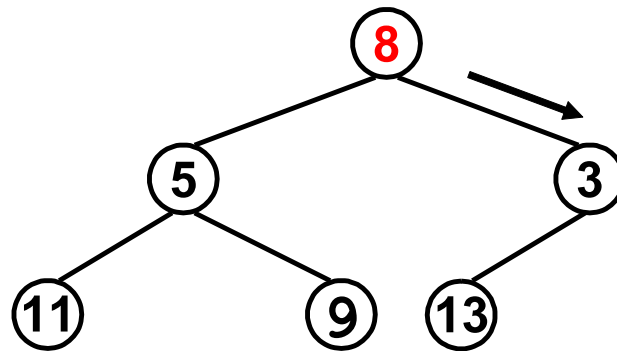
Eliminare din heap (min-heap)



Extras = 2

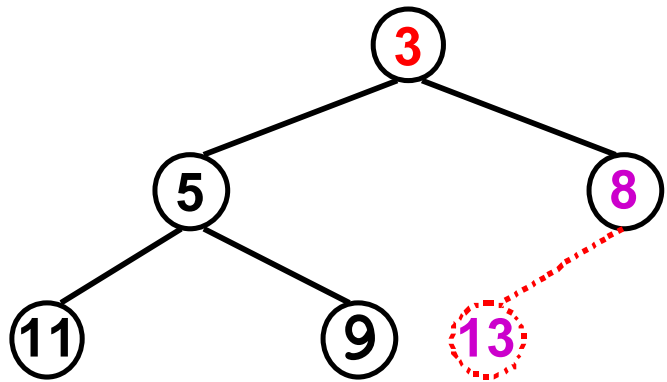
Temp = 8

Elimina
valoarea din
radacina



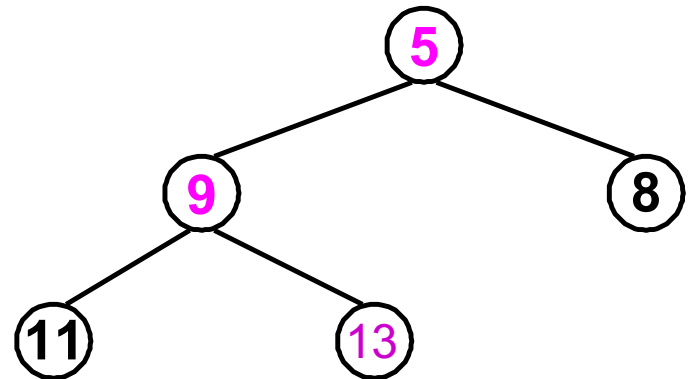
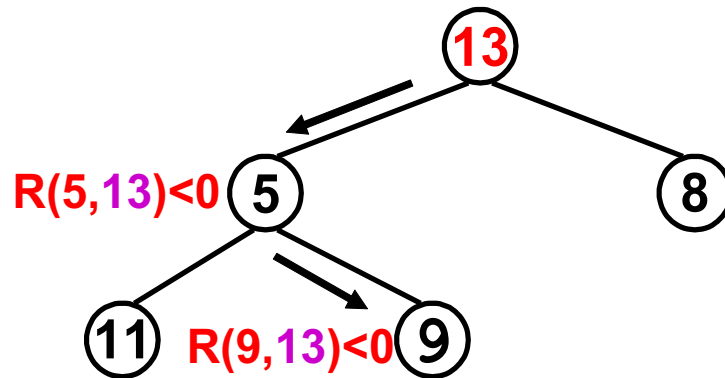
$R(3, 8) < 0$

Eliminare din heap (min-heap)

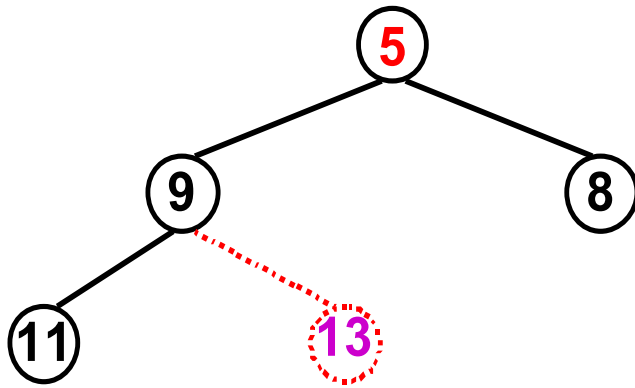


Extras = 3

Temp = 13

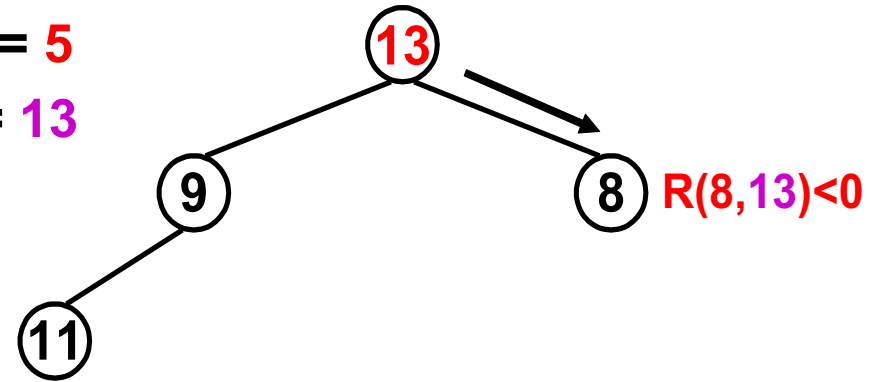


Eliminare din heap (min-heap)

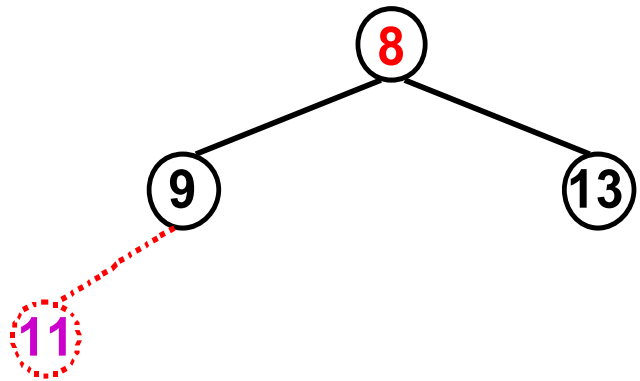


Extras = 5

Temp = 13

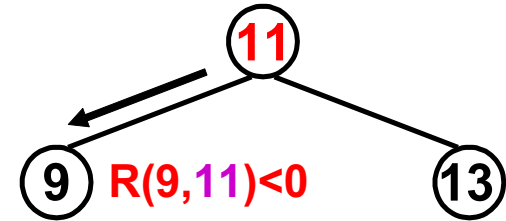


Eliminare din heap (min-heap)

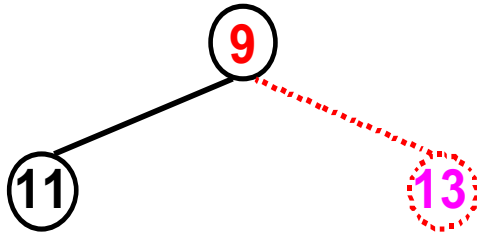


Extras = 8

Temp = 11

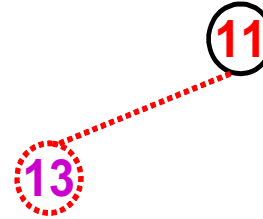


Eliminare din heap (min-heap)



Extras = 9

Temp = 13



Extras = 11

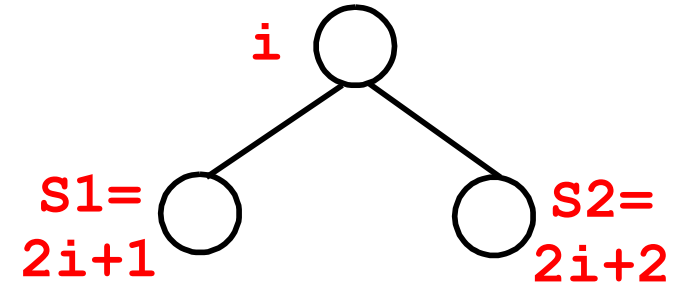
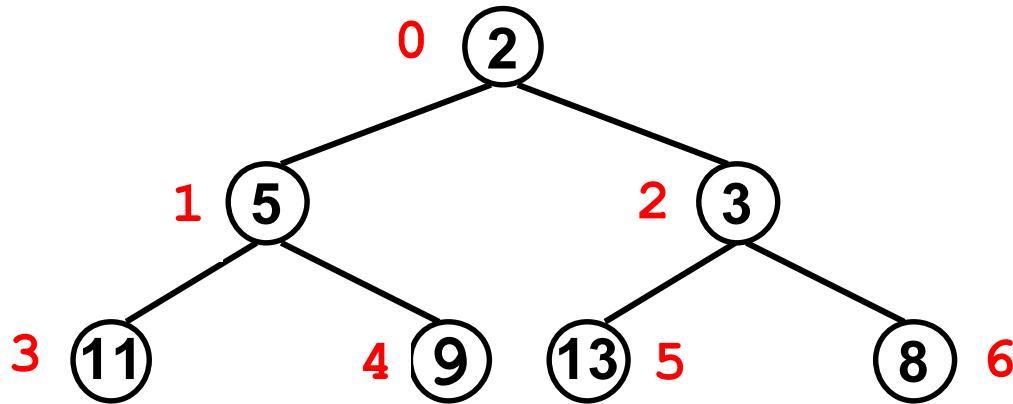
Temp = 13



Algoritm eliminare din heap

```
{ Extras = info(radacina);
  D = radacina;
  T = info(ultimul_nod); info(D) = T;
  elimina ultimul nod;
  cat timp exista fiu_st(D)
  { s = fiu_st;
    daca (exista fiu_dr(D)
          && R(cheie(s), cheie(fiu_dr))) > 0
      atunci s = fiu_dr;
    daca R(cheie(T), cheie(s)) <= 0 atunci iesire
      altfel { interschimba info(D) cu info(s);
              D = s;
            }
  }
  intoarce Extras;
}
```


Reprezentare heap



Indice parinte(i) = $(i-1) / 2$

Indice Subarbore stang(i) = $2*i + 1$

Indice Subarbore stang(i) = $2*i + 1$

Pentru reprezentarea arborelui de selectie se poate utiliza un vector.

In cazul de fata vectorul va contine:

{ 2, 5, 3, 11, 9, 13, 8 }